Engajaa

Cresça nas redes de verdade.

Especificações Completas — Fase 1 (MVP)

Identidade Visual · Produto · Arquitetura · Stack · Claude Code Ready

Versão 1.0 | Abril 2026 | Confidencial

1. Identidade Visual

Todas as definições abaixo são autoritativas. Nenhum elemento deve ser alterado sem aprovação do produto.

1.1 Logo

A logo do Engajaa é composta por dois elementos sobrepostos, sem texto, formando o símbolo da marca:

Elemento 1 — Barras de Gráfico (azul)

- 3 barras retangulares verticais com cantos arredondados no topo (border-radius no topo)
- Crescentes da esquerda para a direita: pequena → média → alta
- Cor: #3B82F6 (Azul Engajaa)
- Posicionadas à direita do ícone

Elemento 2 — Seta / Avião de papel (roxo)

- Forma triangular diagonal, inclinada ~45° para cima-direita
- Sobreposta às barras, ocupando posição frontal (z-index maior)
- Gradiente: #7C3AED → #A855F7 (roxo escuro para lilás)
- Detalhe interno: linha diagonal clara indicando dobra/aba
- Aba inferior: triângulo menor em #9B6FE8 com opacidade 85%

Tipografia do logotipo

Fonte: Peso Black/900, sem serifa moderna (referência: DM Sans Black ou similar)

Cor: #1E1B2E (Dark Engajaa) — nunca colorido

Tamanho relativo: Aprox. 2× a altura do ícone

Espaçamento: letter-spacing: -0.03em

Implementação SVG do ícone

```svg
<svg viewBox="0 0 100 100" fill="none" xmlns="http://www.w3.org/2000/svg">
  <!-- Barras azuis (gráfico) -->
  <rect x="28" y="62" width="14" height="26" rx="4" fill="#3B82F6"/>
  <rect x="46" y="46" width="14" height="42" rx="4" fill="#3B82F6"/>
  <rect x="64" y="30" width="14" height="58" rx="4" fill="#3B82F6"/>
  <!-- Seta roxa (avião de papel) -->
  <path d="M12 55 L52 10 L60 38 L36 46 Z" fill="url(#grad)"/>
  <path d="M36 46 L60 38 L48 68 Z" fill="#9B6FE8" opacity="0.85"/>
  <path d="M22 52 L52 10" stroke="#B48EF0" stroke-width="2.5"
        stroke-linecap="round" opacity="0.5"/>
  <defs>
    <linearGradient id="grad" x1="12" y1="55" x2="60" y2="10"
                    gradientUnits="userSpaceOnUse">
      <stop offset="0%" stop-color="#7C3AED"/>
      <stop offset="100%" stop-color="#A855F7"/>
    </linearGradient>
  </defs>
</svg>
```

⚠️ Nunca substituir o ícone por emoji, imagem rasterizada ou outro símbolo. Sempre usar o SVG acima. Nunca alterar cores, proporções ou gradientes da logo.

1.2 Paleta de Cores

Amostra	Hex	Nome	Uso
	#7C3AED	Roxo Primário	Cor principal da marca. CTA, ícone ativo, links, destaques
	#6366F1	Roxo Médio	Gradientes, hover states, elementos secundários
	#3B82F6	Azul Engajaa	Barras do gráfico na logo, badges, indicadores
	#22C55E	Verde Sucesso	Variações positivas, status 'conectado', crescimento
	#1E1B2E	Dark (fundo)	Background da sidebar, modais escuros, texto principal dark
	#2A2640	Dark Médio	Cards da sidebar, hover na nav
	#353050	Dark Claro	Bordas internas na sidebar, separadores
	#F8F7FF	Off-White	Background geral da aplicação
	#FFFFFF	Branco	Cards de conteúdo, inputs, superfícies principais
	#EBEBF5	Borda	Bordas sutis de cards e separadores no tema claro
	#6B7280	Cinza Texto	Textos secundários, labels, metadados
	#111827	Texto Principal	Títulos e corpo de texto sobre fundos claros

Gradientes de uso

- Primário: linear-gradient(135deg, #7C3AED, #3B82F6) — botões CTA, score card, elementos de destaque
- Dark: linear-gradient(135deg, #1E1B2E, #2A2640) — cards escuros, sugestão do dia
- Roxo sutil: linear-gradient(135deg, #7C3AED22, #6366F122) — nav item ativo na sidebar

1.3 Tipografia

Elemento	Fonte	Peso / Tamanho	Cor
Logo wordmark	DM Sans / similar Black	900 / escala livre	#1E1B2E
Título de página	DM Sans	900 / 26px	#111827
Heading card	DM Sans	800 / 16px	#111827
Subtítulo	DM Sans	400 / 14px	#6B7280
Label uppercase	DM Sans	700 / 11px	#6B7280
Métrica destaque	DM Sans	900 / 28px	#111827
Body text	DM Sans	400 / 13–14px	#6B7280
Badge / tag	DM Sans	700–800 / 10–12px	variável
Código / tokens	Courier New / mono	400 / 12–14px	#A5B4FC

ℹ️ Fonte primária: DM Sans (Google Fonts). Fallback: 'Helvetica Neue', sans-serif. Importar via: @import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700;800;900&display=swap')

1.4 Componentes de UI — Padrões

Cards

- Background: #FFFFFF
- Border: 1px solid #EBEBF5
- Border-radius: 20px (cards maiores), 14–16px (cards menores)
- Padding: 20–24px
- Box-shadow: nenhum por padrão; hover: 0 4px 24px rgba(124,58,237,0.08)

Botão Primário (CTA)

- Background: linear-gradient(135deg, #7C3AED, #3B82F6)
- Texto: #FFFFFF, font-weight 700, font-size 13–14px
- Border-radius: 12–14px
- Padding: 10–14px 20px
- Border: none
- Hover: brightness(1.08), transform: translateY(-1px)

Botão Secundário / Outline

- Background: transparent
- Border: 1.5px solid [cor contextual]
- Texto: [cor contextual], font-weight 700
- Border-radius: 12px

Badges / Tags

- Formato: background = cor + '18' (hex com 18 = ~10% opacidade), texto na cor cheia
- Border-radius: 6–8px, padding: 2–4px 8–10px
- Font-size: 10–12px, font-weight: 700–800

Sidebar

- Background: #1E1B2E (Dark)
- Largura: 220px, fixed/sticky
- Item ativo: fundo gradiente roxo sutil + barra lateral esquerda 3px gradiente
- Item hover: background #2A2640
- Separadores internos: border 1px solid #353050

2. Especificações de Produto — Fase 1

Nome: Engajaa

Tagline: Cresça nas redes de verdade.

Público-alvo: Criadores de conteúdo e marcas que querem potencializar resultados no Instagram

Modelo: SaaS multi-tenant. Cada tenant = uma empresa/conta com múltiplos usuários

Foco Fase 1: Análise do perfil próprio do usuário (dados reais via Instagram Graph API)

🚫 Fora do escopo da Fase 1: análise de concorrentes, benchmark de nicho, outras redes sociais (TikTok, YouTube, LinkedIn). Esses itens são Fase 2.

2.1 Módulos da Fase 1

#	Módulo	Descrição
1	Dashboard	Visão geral: métricas principais, top posts, gráfico de crescimento, sugestão do dia
2	Análises	Gráficos por período e por formato (Reels, Carrossel, Feed, Stories). Melhores horários.
3	Sugestões IA	Cards com sugestões de conteúdo geradas por IA baseadas nos dados reais

#	Módulo	Descrição
		do perfil
4	Agendamento	Calendário semanal. Criação, edição e listagem de publicações agendadas
5	Configurações	Conexão Instagram OAuth, gestão da equipe (roles), plano e billing

2.2 Módulo 1 — Dashboard

Componentes obrigatórios

- Saudação personalizada: 'Olá, [nome]!' com data/contexto
- 4 cards de métricas: Novos Seguidores, Likes Obtidos, Vídeos Enviados, Taxa de Engajamento
- Cada card: valor destacado + variação % vs mês anterior (verde/vermelho) + mini ícone
- Gráfico de crescimento de seguidores: linha/área com gradiente roxo (últimas 12 semanas)
- Painel 'Insights do Conteúdo': top 3 formatos com barra de score colorida
- Card 'Sugestão do Dia': fundo dark, texto destacado, botão CTA gradiente

Dados exibidos (via Instagram Graph API)

- Seguidores: followers_count
- Likes: soma de like_count dos posts do período
- Engajamento: (likes + comments + saves) / reach × 100
- Crescimento semanal: diferença de followers_count entre semanas

2.3 Módulo 2 — Análises

- Seletor de período: 7 dias / 30 dias / 90 dias
- Gráfico de barras: Alcance diário (últimos 7 dias)
- Gráfico de barras: Taxa de engajamento diária
- Tabela de performance por formato: Reels, Carrossel, Feed, Stories
  - Colunas: Formato | Alcance | Engajamento | Salvamentos | Score
  - Melhor formato destacado com card dark + badge 'TOP'
- Melhores horários para postagem: top 3 janelas com barra de intensidade

2.4 Módulo 3 — Sugestões com IA

- Grid 2 colunas de cards clicáveis
- Cada card contém: ícone temático, tipo de conteúdo (badge), tag de objetivo, título, insight explicativo
- Tags de objetivo: Alto Impacto (verde), Engajamento (roxo), Conversão (azul), Audiência (amarelo)
- Ao clicar: expande com botões 'Criar conteúdo' e 'Agendar'
- Sugestões geradas por IA baseadas em: melhor formato do perfil, horários de pico, histórico de engajamento

🤖 Integração IA: chamar Claude API (claude-sonnet-4-20250514) com contexto do perfil (métricas reais,

formatos, horários) e retornar sugestões estruturadas em JSON com campos: type, title, insight, tag, icon.

2.5 Módulo 4 — Agendamento

- Calendário semanal com pills coloridos por tipo de conteúdo
- Lista de publicações: dia, horário, tipo, status (agendado | rascunho | publicado | falhou)
- Status visual: agendado = verde, rascunho = amarelo, falhou = vermelho
- Botão '+ Nova publicação' abre modal com: tipo, legenda, upload de mídia, data/hora
- Job worker publica automaticamente no horário agendado via Instagram Graph API

2.6 Módulo 5 — Configurações

Aba: Perfil conectado

- Status da conexão Instagram (conectado / desconectado)
- Botão 'Conectar Instagram' inicia fluxo OAuth
- Exibe: @handle, foto de perfil, data de conexão, validade do token

Aba: Equipe

- Lista de usuários do tenant com role e status
- Botão 'Convidar usuário' (envia e-mail com link de aceitação)
- Alterar role de usuário (somente owner/admin)
- Remover usuário do tenant

Aba: Plano

- Plano atual, valor, data de renovação
- Botão 'Gerenciar' redireciona para portal de billing (Stripe)

3. Arquitetura Multi-Tenant

Estratégia: Row-Level Security com tenant_id em todas as tabelas de negócio

Identificação: JWT com payload { userId, tenantId, role } + middleware de tenant na API

Isolamento: Toda query DEVE incluir WHERE tenant_id = :tenantId. RLS Policy ativa no Postgres.

⚠️ Regra crítica: nunca confiar apenas na camada de aplicação para isolamento. Ativar RLS Policy no Postgres como segunda linha de defesa obrigatória.

3.1 Stack Tecnológica

Camada	Tecnologia	Observações
Backend API	Node.js + Express	Arquitetura em camadas: routes → controllers → services → repositories
Banco de dados	PostgreSQL	Row-Level Security ativo. UUID como PK. Índice obrigatório em tenant_id
ORM	Prisma ou Knex	Prisma recomendado para type safety. Migrations versionadas
Frontend	React + Vite	SPA. React Context para tenant/auth state. Axios com interceptor JWT
Auth	JWT RS256	Access token 15min (memória). Refresh token 7d (cookie HttpOnly)
Jobs / Queue	BullMQ + Redis	Sync Instagram, publicação agendada, renovação de tokens
Instagram API	Instagram Graph API v18+	OAuth 2.0. Permissões: basic, insights, content_publish
IA / Sugestões	Anthropic Claude API	claude-sonnet-4-20250514. Chamada server-side com contexto do perfil
Armazenamento	AWS S3 ou Cloudflare R2	Upload de mídias para publicação agendada
E-mail	Resend ou SendGrid	Convites de equipe, alertas de token expirado
Billing	Stripe	Planos, assinaturas, portal de billing

3.2 Estrutura de Pastas — Backend

```
engajaa-api/
├── src/
│   ├── config/           # env, database, redis
│   ├── middlewares/
│   │   ├── auth.js       # Valida JWT, popula req.user
│   │   ├── tenant.js     # Extrai tenantId, injeta em req.tenantId
│   │   └── roles.js      # Guard de role por rota
│   ├── modules/
│   │   ├── auth/         # login, refresh, logout, invite
│   │   ├── instagram/    # OAuth, sync, webhook
│   │   ├── analytics/    # Overview, posts, best-times
│   │   ├── suggestions/  # Geração IA, cache
│   │   ├── scheduler/    # CRUD de agendamentos
│   │   └── users/        # CRUD usuários, roles
│   ├── jobs/
│   │   ├── syncInstagram.js    # Sync a cada 6h
│   │   ├── publishPost.js      # Publica no horário agendado
│   │   └── refreshTokens.js    # Renova tokens IG (55d)
│   ├── lib/              # Clientes: anthropic, instagram, s3
│   └── app.js
├── prisma/
│   ├── schema.prisma
│   └── migrations/
└── package.json
```

3.3 Estrutura de Pastas — Frontend

```
engajaa-web/
├── src/
│   ├── assets/
│   │   └── logo.svg          # SVG da logo (ícone + wordmark)
│   ├── components/
│   │   ├── ui/               # Button, Badge, Card, Modal, Input
│   │   ├── layout/           # Sidebar, Header, PageWrapper
│   │   └── charts/           # LineChart, BarChart, SparkLine
│   ├── modules/
│   │   ├── dashboard/
│   │   ├── analytics/
│   │   ├── suggestions/
│   │   ├── scheduler/
│   │   └── settings/
│   ├── contexts/
│   │   ├── AuthContext.jsx    # User, token, login/logout
│   │   └── TenantContext.jsx  # tenantId, plan, igAccount
│   ├── services/
│   │   └── api.js            # Axios instance + interceptors
│   ├── styles/
│   │   ├── tokens.css        # CSS custom properties da brand
│   │   └── global.css
│   └── App.jsx
└── package.json
```

3.4 CSS Design Tokens

```
:root {
  /* Cores */
  --color-purple:       #7C3AED;
  --color-purple-mid:   #6366F1;
  --color-blue:         #3B82F6;
  --color-green:        #22C55E;
  --color-dark:         #1E1B2E;
  --color-dark-mid:     #2A2640;
  --color-dark-light:   #353050;
  --color-off-white:    #F8F7FF;
  --color-white:        #FFFFFF;
  --color-border:       #EBEBF5;
  --color-gray:         #6B7280;
  --color-text:         #111827;

  /* Gradientes */
  --gradient-primary:   linear-gradient(135deg, #7C3AED, #3B82F6);
  --gradient-dark:      linear-gradient(135deg, #1E1B2E, #2A2640);
  --gradient-subtle:    linear-gradient(135deg, #7C3AED22, #6366F122);

  /* Tipografia */
  --font-sans:          'DM Sans', 'Helvetica Neue', sans-serif;
  --font-mono:          'Courier New', monospace;

  /* Raios */
```

```
  --radius-sm:          8px;
  --radius-md:          12px;
  --radius-lg:          16px;
  --radius-xl:          20px;

  /* Sombras */
  --shadow-card:        0 1px 3px rgba(0,0,0,0.06);
  --shadow-hover:       0 4px 24px rgba(124,58,237,0.10);
  --shadow-modal:       0 20px 60px rgba(0,0,0,0.20);
}
```

4. Banco de Dados — PostgreSQL

4.1 Schema Prisma Completo

```
// prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Tenant {
  id          String   @id @default(uuid())
  name        String
  plan        Plan     @default(FREE)
  status      TenantStatus @default(ACTIVE)
  createdAt   DateTime @default(now())
  users       User[]
  igAccounts  InstagramAccount[]
  igPosts     IgPost[]
  scheduled   ScheduledPost[]
}

model User {
  id           String   @id @default(uuid())
  tenantId     String
  tenant       Tenant   @relation(fields:[tenantId], references:[id])
  email        String
  passwordHash String
  role         Role     @default(VIEWER)
  lastLoginAt  DateTime?
  createdAt    DateTime @default(now())
  @@unique([tenantId, email])
  @@index([tenantId])
}

model InstagramAccount {
  id             String   @id @default(uuid())
  tenantId       String
  tenant         Tenant   @relation(fields:[tenantId], references:[id])
```

```
  igUserId       String
  username       String
  accessToken    String   // criptografado AES-256
  tokenExpiresAt DateTime
  connectedAt    DateTime @default(now())
  @@index([tenantId])
}

model IgPost {
  id            String   @id @default(uuid())
  tenantId      String
  tenant        Tenant   @relation(fields:[tenantId], references:[id])
  igMediaId     String
  mediaType     MediaType
  caption       String?
  likeCount     Int      @default(0)
  commentsCount Int      @default(0)
  reach         Int      @default(0)
  impressions   Int      @default(0)
  saved         Int      @default(0)
  timestamp     DateTime
  syncedAt      DateTime @default(now())
  @@index([tenantId])
  @@index([tenantId, timestamp])
}

model ScheduledPost {
  id          String   @id @default(uuid())
  tenantId    String
  tenant      Tenant   @relation(fields:[tenantId], references:[id])
  createdById String
  mediaType   PostType
  caption     String?
  mediaUrls   Json     // string[]
  scheduledAt DateTime
  status      PostStatus @default(DRAFT)
  createdAt   DateTime @default(now())
  @@index([tenantId])
  @@index([tenantId, scheduledAt])
}

enum Plan         { FREE PRO ENTERPRISE }
enum TenantStatus { ACTIVE SUSPENDED CANCELLED }
enum Role         { OWNER ADMIN EDITOR VIEWER }
enum MediaType    { IMAGE VIDEO CAROUSEL_ALBUM }
enum PostType     { REELS CAROUSEL IMAGE STORY }
enum PostStatus   { DRAFT SCHEDULED PUBLISHED FAILED }
```

4.2 Row-Level Security — PostgreSQL

```
-- Ativar RLS em todas as tabelas de negócio
ALTER TABLE "User"             ENABLE ROW LEVEL SECURITY;
ALTER TABLE "InstagramAccount" ENABLE ROW LEVEL SECURITY;
ALTER TABLE "IgPost"           ENABLE ROW LEVEL SECURITY;
ALTER TABLE "ScheduledPost"    ENABLE ROW LEVEL SECURITY;

-- Policy: usuário só acessa registros do próprio tenant
```

```
CREATE POLICY tenant_isolation ON "User"
  USING (tenant_id = current_setting('app.tenant_id')::uuid);

CREATE POLICY tenant_isolation ON "IgPost"
  USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- Aplicar no início de cada transação (no middleware Node):
-- SET LOCAL app.tenant_id = '<tenantId>';
```

5. API — Endpoints Fase 1

Método	Endpoint	Role	Descrição
POST	/auth/login	público	Autenticação. Retorna access + refresh token
POST	/auth/refresh	público	Renova access token via refresh cookie
POST	/auth/logout	autenticado	Invalida refresh token
POST	/auth/invite/accept	público	Aceita convite e cria senha
GET	/instagram/connect	owner	Inicia OAuth — redireciona para Meta
GET	/instagram/callback	owner	Callback OAuth. Salva token criptografado
POST	/instagram/sync	admin	Força sync manual de posts e métricas
GET	/instagram/status	viewer	Status da conexão e dados do perfil IG
GET	/analytics/overview	viewer	Métricas gerais do período selecionado
GET	/analytics/posts	viewer	Lista de posts com métricas individuais
GET	/analytics/by-format	viewer	Performance agrupada por tipo de mídia
GET	/analytics/best-times	viewer	Top 3 janelas de horário para postagem
GET	/suggestions	editor	Lista sugestões IA (com cache de 24h)
POST	/suggestions/refresh	editor	Força geração de novas sugestões via IA
GET	/scheduler/posts	viewer	Lista publicações agendadas do tenant
POST	/scheduler/posts	editor	Cria novo agendamento de publicação
PATCH	/scheduler/posts/:id	editor	Atualiza agendamento (data, legenda, status)
DELETE	/scheduler/posts/:id	editor	Remove agendamento (somente draft/scheduled)
GET	/users	admin	Lista usuários do tenant
POST	/users/invite	admin	Envia convite por e-mail
PATCH	/users/:id/role	owner	Altera role de um usuário
DELETE	/users/:id	owner	Remove usuário do tenant

Método	Endpoint	Role	Descrição
GET	/tenant/plan	owner	Dados do plano atual
POST	/tenant/billing-portal	owner	Gera URL do portal de billing Stripe

6. Integração Instagram Graph API

6.1 Fluxo OAuth Completo

- Usuário clica em 'Conectar Instagram' no painel
- Backend redireciona para: https://api.instagram.com/oauth/authorize com client_id, redirect_uri, scope
- Meta exibe tela de autorização ao usuário
- Meta redireciona para /instagram/callback?code=XXX
- Backend troca code por short-lived token (validade: 1h)
- Backend troca short-lived por long-lived token (validade: 60 dias)
- Access token criptografado com AES-256 e salvo em InstagramAccount
- Job agendado (a cada 55 dias) renova o long-lived token automaticamente

6.2 Permissões (scopes) Necessárias

Permissão	Finalidade
instagram_basic	Dados do perfil: username, media básico
instagram_manage_insights	Alcance, impressões, engajamento, salvamentos
instagram_content_publish	Publicação de posts via API
pages_read_engagement	Necessária para acessar insights (requisito Meta)

6.3 Estratégia de Sync

- Sync automático a cada 6 horas via BullMQ job
- Dados armazenados em cache local (tabela IgPost no Postgres)
- Dashboard exibe dados do cache com timestamp do último sync visível
- Insights de alcance/impressões têm delay natural de 24–48h na API do Meta — comunicar ao usuário

> ℹ️ A Instagram Graph API não permite acesso a dados de outros perfis. Todas as métricas da Fase 1 são exclusivamente do perfil conectado pelo tenant. Análise de concorrentes está prevista somente na Fase 2.

7. Segurança — Checklist MVP

	Item de segurança
✓	Access token Instagram criptografado AES-256 em repouso no banco
✓	JWT com algoritmo RS256 (chaves assimétricas, nunca HS256)
✓	Access token em memória JS (nunca localStorage). Refresh em cookie HttpOnly + Secure + SameSite=Strict
✓	Row-Level Security ativo no Postgres + SET LOCAL app.tenant_id no middleware
✓	Rate limiting por tenant: 100 req/min. Global: 1000 req/min por IP
✓	Validação de input com Zod em todos os endpoints (nunca confiar no cliente)
✓	HTTPS obrigatório em staging e produção. HSTS header ativo
✓	Variáveis de ambiente via vault seguro (Doppler, AWS Secrets Manager ou similar)
✓	Logs de auditoria para: connect/disconnect IG, alterar role, deletar usuário, publicar post
✓	Headers de segurança: CSP, X-Frame-Options, X-Content-Type-Options via Helmet.js
✓	CORS configurado para aceitar apenas origens da aplicação (nunca *)

8. Roles e Permissões

Role	Permissões
OWNER	Acesso total. Gerencia plano e billing. Conecta/desconecta Instagram. Cria, edita e remove qualquer usuário. Altera roles.
ADMIN	Gerencia usuários (exceto alterar role de outros admins). Acessa todos os dados, métricas e configurações.
EDITOR	Cria e agenda publicações. Acessa sugestões IA. Visualiza analytics. Não gerencia usuários.
VIEWER	Somente leitura. Visualiza dashboard, analytics e publicações agendadas. Não pode criar nem editar.

9. Ambiente Local — Docker Compose

O Engajaa roda inteiramente em containers Docker locais via Docker Compose. Nenhum servico precisa estar instalado diretamente na maquina — apenas Docker Desktop.

9.1 Servicos do Docker Compose

Servico	Imagem	Porta	Descricao
api	node:20-alpine (build)	3000:3000	API Node.js + Express. Hot-reload nodemon
web	node:20-alpine (build)	5173:5173	Frontend React + Vite. HMR ativo
postgres	postgres:16-alpine	5432:5432	Banco principal. Dados em volume persistente
redis	redis:7-alpine	6379:6379	Fila BullMQ + cache
worker	node:20-alpine (build)	—	BullMQ workers: sync IG, publicar posts, renovar tokens
maildev	maildev/maildev	1080:1080	UI de e-mail local. Captura convites sem enviar real

9.2 docker-compose.yml

```
version: '3.9'

services:

  postgres:
    image: postgres:16-alpine
    container_name: engajaa_postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB:       engajaa_dev
      POSTGRES_USER:     engajaa
      POSTGRES_PASSWORD: engajaa_secret
    ports:
      - '5432:5432'
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U engajaa -d engajaa_dev']
      interval: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: engajaa_redis
    ports:
      - '6379:6379'
    volumes:
      - redis_data:/data
    healthcheck:
      test: ['CMD', 'redis-cli', 'ping']
      interval: 5s
      retries: 5

  api:
    build:
      context: ./engajaa-api
      dockerfile: Dockerfile.dev
    container_name: engajaa_api
    ports:
```

```
      - '3000:3000'
    environment:
      NODE_ENV:          development
      DATABASE_URL:      postgresql://engajaa:engajaa_secret@postgres:5432/engajaa_dev
      REDIS_URL:         redis://redis:6379
      JWT_PRIVATE_KEY: ${JWT_PRIVATE_KEY}
      JWT_PUBLIC_KEY: ${JWT_PUBLIC_KEY}
      AES_SECRET: ${AES_SECRET}
      IG_CLIENT_ID: ${IG_CLIENT_ID}
      IG_CLIENT_SECRET: ${IG_CLIENT_SECRET}
      IG_REDIRECT_URI:   http://localhost:3000/instagram/callback
      ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY}
    volumes:
      - ./engajaa-api:/app
      - /app/node_modules
    depends_on:
      postgres: { condition: service_healthy }
      redis:    { condition: service_healthy }
    command: npx nodemon src/app.js

  worker:
    build:
      context: ./engajaa-api
      dockerfile: Dockerfile.dev
    container_name: engajaa_worker
    environment:
      NODE_ENV:          development
      DATABASE_URL:      postgresql://engajaa:engajaa_secret@postgres:5432/engajaa_dev
      REDIS_URL:         redis://redis:6379
      AES_SECRET: ${AES_SECRET}
      IG_CLIENT_ID: ${IG_CLIENT_ID}
      IG_CLIENT_SECRET: ${IG_CLIENT_SECRET}
      ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY}
    volumes:
      - ./engajaa-api:/app
      - /app/node_modules
    depends_on:
      postgres: { condition: service_healthy }
      redis:    { condition: service_healthy }
    command: node src/jobs/worker.js

  web:
    build:
      context: ./engajaa-web
      dockerfile: Dockerfile.dev
    container_name: engajaa_web
    ports:
      - '5173:5173'
    environment:
      VITE_API_URL: http://localhost:3000
    volumes:
      - ./engajaa-web:/app
      - /app/node_modules
    depends_on:
      - api

  maildev:
    image: maildev/maildev
    container_name: engajaa_maildev
    ports:
      - '1080:1080'
      - '1025:1025'
```

```
volumes:
  postgres_data:
  redis_data:
```

9.3 Dockerfiles de Desenvolvimento

engajaa-api/Dockerfile.dev

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "src/app.js"]
```

engajaa-web/Dockerfile.dev

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 5173
CMD ["npx", "vite", "--host", "0.0.0.0"]
```

9.4 Arquivo .env (raiz do projeto)

```
# .env — NUNCA commitar. Adicionar ao .gitignore

# JWT RS256 (gerar: openssl genrsa -out private.pem 2048)
JWT_PRIVATE_KEY="-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----"
JWT_PUBLIC_KEY="-----BEGIN PUBLIC KEY-----\n...\n-----END PUBLIC KEY-----"

# AES-256 para tokens Instagram (exatamente 32 chars)
AES_SECRET=sua_chave_aes_256_bits_aqui_32ch

# Meta / Instagram App
IG_CLIENT_ID=seu_instagram_client_id
IG_CLIENT_SECRET=seu_instagram_client_secret

# Anthropic
ANTHROPIC_API_KEY=sk-ant-...

# E-mail (aponta para maildev em dev)
SMTP_HOST=maildev
SMTP_PORT=1025
```

9.5 Estrutura do Repositorio (monorepo)

```
engajaa/                    # raiz do monorepo
├── docker-compose.yml
├── .env                    # variaveis locais (nao commitar)
├── .env.example            # template sem valores reais
├── .gitignore
├── engajaa-api/            # Backend Node.js
│   ├── Dockerfile.dev
│   ├── package.json
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── migrations/
│   └── src/
└── engajaa-web/            # Frontend React
    ├── Dockerfile.dev
    ├── package.json
    ├── vite.config.js
    └── src/
```

9.6 Comandos do dia a dia

Comando	O que faz
docker compose up -d	Sobe todos os containers em background
docker compose up -d --build	Rebuild das imagens + sobe os containers
docker compose down	Para e remove containers (dados preservados)
docker compose down -v	Para + remove containers E volumes (reset total)
docker compose logs -f api	Logs em tempo real da API
docker compose logs -f worker	Logs em tempo real dos jobs BullMQ
docker compose exec api npx prisma migrate dev	Roda migrations dentro do container
docker compose exec api npx prisma studio	Abre Prisma Studio para inspecionar o banco
docker compose exec api npx prisma db seed	Popula banco com dados de seed
docker compose restart api	Reinicia somente o container da API

!

Acessos locais: App http://localhost:5173 | API http://localhost:3000 | MailDev http://localhost:1080 | Prisma Studio http://localhost:5555

10. Prompt Inicial para Claude Code

Cole o prompt abaixo ao iniciar o projeto no Claude Code:

```
Você está construindo o Engajaa — plataforma SaaS multi-tenant
para crescimento no Instagram.

# STACK
- Backend: Node.js + Express + Prisma + PostgreSQL
- Frontend: React + Vite + CSS custom properties
- Auth: JWT RS256 (access 15min memória, refresh 7d cookie HttpOnly)
- Jobs: BullMQ + Redis
- IA: Anthropic Claude API (claude-sonnet-4-20250514)

# IDENTIDADE VISUAL
- Fonte: DM Sans (Google Fonts)
- Roxo primário: #7C3AED
- Azul: #3B82F6
- Verde: #22C55E
- Dark sidebar: #1E1B2E
- Background: #F8F7FF
- Gradiente CTA: linear-gradient(135deg, #7C3AED, #3B82F6)
- Sidebar: 220px, dark, com nav items e score card
- Cards: border-radius 20px, border 1px solid #EBEBF5

# MULTI-TENANT
- Isolamento: tenant_id em todas as tabelas + RLS Policy no Postgres
- JWT payload: { userId, tenantId, role }
- Middleware de tenant injeta tenantId em cada request

# FASE 1 — MÓDULOS
1. Dashboard (métricas, gráfico crescimento, sugestão do dia)
2. Análises (por período, por formato, melhores horários)
3. Sugestões IA (via Claude API com contexto do perfil)
4. Agendamento (calendário + lista + publicação automática)
5. Configurações (OAuth Instagram, equipe, plano)

Siga o documento de especificações completo.
Comece pela estrutura do projeto e configuração do banco.
```

10. Roadmap

Fase	Status	Escopo
Fase 1	Em build	Multi-tenant base, OAuth Instagram, dashboard de métricas, sugestões IA, agendamento
Fase 2	Planejado	Growth strategy: benchmark por nicho, análise de concorrentes, estratégias comparativas, score de crescimento
Fase 3	Futuro	TikTok, YouTube, LinkedIn. Relatórios avançados. White-label. API pública.

— Documento gerado para uso no Claude Code —